

CSC3066

Deep Learning

*Implementing Artificial Neural Networks in
Keras*

Anna Jurek-Loughrey

What's to come today

- ❑ Deep Learning Frameworks
- ❑ Building Deep ANNs with Keras

Deep Learning Libraries

- ❑ TensorFlow
 - Library created by Google used for Machine Learning applications
 - A framework that can live on it's own
- ❑ PyTorch
 - Machine Learning library created by Facebook
 - A framework that can live on it's own
- ❑ Keras
 - High-level API that sits on top of other deep Learning frameworks
 - It is designed to enable fast experimentation with DNNs
 - Currently it supports TensorFlow, Theano, and CNTK.

Keras

- ❑ **Keras** is an Open Source Neural Network library written in Python that runs on top of Theano or TensorFlow.
- ❑ It is designed to be modular, fast and easy to use.
- ❑ Keras doesn't handle low-level computation. Instead, it uses another library to do it, called the “backend”.
- ❑ Keras is not a framework on it's own.

Keras vs TensorFlow

Keras	TensorFlow
High-Level API Wrapper	Low-Level API
Easy to use if you know Python language	You need to learn the syntax of using some of TensorFlow functions
Rapid deployment for making model with standard layers	Allows you to make an arbitrary computational graph or model layers
The main focus of the library is on easy use. Clean and simple, allowing standard deep learning models to be defined, fit, and evaluated in just a few lines of code.	Focus of this library is on research, flexibility, and speed, not ease of use.

tf.Keras

- ❑ In 2019, Google released a new version of their **TensorFlow** deep learning library (TensorFlow 2) that integrated the **Keras** API directly and promoted this interface as the default or standard interface for deep learning development on the platform.
- ❑ This integration is commonly referred to as the tf.keras interface or API (“tf” is short for “TensorFlow”). This is to distinguish it from the so-called standalone **Keras** open source project.

Keras vs tf.Keras

- ❑ Standalone Keras: the standalone open source project that supports TensorFlow, Theano and CNTK backend.
- ❑ tf.keras: the Keras API integrated into TensorFlow 2.
- ❑ First, the TensorFlow module is imported and named “tf”; then, Keras API elements are accessed via calls to tf.keras

```
import keras  
  
model = keras.Sequential()
```

```
import tensorflow as tf  
  
model = tf.keras.Sequential()
```

Neural Network:

Building ANNs with Keras

Deep Learning Model Life-Cycle

□ The 5-Step Model Life-Cycle

- Define the model
- Compile the model
- Fit (train) the model
- Evaluate the model
- Make predictions

Deep Learning Model Life-Cycle

□ Defining model

- Select the type of model (e.g. MLP, CNN, RNN)
- Choose the architecture of the Neural Network
 - Define the layers of the model
 - Configure each layer with a number of nodes and activation function
 - Connect the layers together into a cohesive model

Deep Learning Model Life-Cycle

□ Defining model (select model type)

- E.g. model type: **Sequential**
 - The Sequential model is a linear stack of layers

```
#import tensorflow
import tensorflow as tf
#import keras
from tensorflow import keras
from keras.models import Sequential
```

```
model = Sequential()
```

Deep Learning Model Life-Cycle

□ Defining model (Choose the architecture of the NN)

- Defining the layers of the model
 - E.g. layer type: **Dense**
 - Fully connected layers are defined using the Dense class.

```
#import tensorflow
import tensorflow as tf
#import keras
from tensorflow import keras
from keras.models import Sequential
from keras.layers import Dense
```

Deep Learning Model Life-Cycle

❑ Defining model (Choose the architecture of the NN)

- Configuring **Dense** layer with a number of nodes and activation function.
 - The number of neurons is specified as the first argument.
 - The activation function can be specified using the **activation** argument.
 - Weights/bias initializers are specified with **kernel_initializer** and **bias_initializer**

Deep Learning Model Life-Cycle

□ Defining Layers Example:

- The input data is of dimension 10 (the `input_dim=10` argument)
- Two hidden layers with 12 and 8 nodes and ReLu activation function
- One neuron in the output layer with Sigmoid activation function
- Weights initialized with uniform distribution and bias with zero values

```
model.add(Dense(12, input_dim=10, activation='relu', kernel_initializer='random_uniform', bias_initializer='zeros'))  
model.add(Dense(8, activation='relu', kernel_initializer='random_uniform', bias_initializer='zeros'))  
model.add(Dense(1, activation='sigmoid', kernel_initializer='random_uniform', bias_initializer='zeros'))
```

Deep Learning Model Life-Cycle

□ The 5-Step Model Life-Cycle

- Define the model
- Compile the model
- Fit (train) the model
- Evaluate the model
- Make predictions

Deep Learning Model Life-Cycle

□ Compiling the model

- Now that the model is defined, we can compile it.
- Compiling the model uses the backend (e.g. TensorFlow or Theano).
- The backend automatically chooses the best way to represent the network for training and making predictions.

Deep Learning Model Life-Cycle

□ Compiling the model

- During compilation, we need to specify some additional properties required when training the network.
 - The **loss function** to use to evaluate the model (set of parameters)
 - The **optimizer** to use to search through different parameters values for the network
 - Any optional **metrics** we want to collect and report during training

Deep Learning Model Life-Cycle

❏ Compiling the model

- When compiling the model, we need to specify some additional properties required when training the network.
 - **Cross entropy** for binary classification problem, defined via `loss` argument.
 - **Gradient Descent optimizer**, defined via the `optimizer` argument
 - **Classification accuracy**, defined via the `metric` argument

Deep Learning Model Life-Cycle

□ Compiling the model

- When compiling the model, we need to specify some additional properties required when training the network.

```
model.compile(loss = 'binary_crossentropy', optimizer = 'sgd', metrics = ['acc'])
```

Deep Learning Model Life-Cycle

□ The 5-Step Model Life-Cycle

- Define the model
- Compile the model
- Fit (train) the model
- Evaluate the model
- Make predictions

Deep Learning Model Life-Cycle

❏ Fitting (training) the model

- Now that we defined our model and compiled it, we can execute it on some data.
- Model can be trained on labeled data by calling the `fit` method on the model.
- Fitting the model requires that you first select the training configuration, such as the **number of epochs** and the **batch size**.

Deep Learning Model Life-Cycle

❏ Fitting (training) the model (sample vs. batch vs. epoch)

- **Sample:** single instance (row of data).
- A sample contains inputs that are fed into the algorithm and an output that is used to compare with prediction and calculate an error.

Deep Learning Model Life-Cycle

❏ Fitting (training) the model (sample vs. batch vs. epoch)

- **Batch:** group of samples that are worked through the network before updating the internal model parameters
- At the end of the batch, the predictions are compared to the expected output values and an error is calculated.
- A train dataset can be divided into one or more batches.
 - Batch Gradient Descent. Batch Size = Size of Training Set
 - Mini-Batch Gradient Descent. $1 < \text{Batch Size} < \text{Size of Training Set}$
 - Stochastic Gradient Descent. Batch Size = 1

Deep Learning Model Life-Cycle

- ❑ Fitting (training) the model (sample vs. batch vs. epoch)
 - **Epoch:** one epoch means that each sample in the train dataset had an opportunity to update the internal model parameters.
 - The number of epochs is a hyperparameter that defines the number of times that the learning algorithm will work through the entire train dataset.

Deep Learning Model Life-Cycle

□ Fitting (training) the model

- The number of epochs is traditionally large, allowing the learning algorithm to run until the error from the model has been sufficiently minimized.
- It is common to create line plots that show epochs along the x-axis as time and the error of the model on the y-axis.
- These plots can help to diagnose whether the model has over learned, under learned, or is suitably fit to the training dataset.

Deep Learning Model Life-Cycle

Fitting (training) the model

```
In [43]: model.compile(loss = 'binary_crossentropy', optimizer = 'sgd', metrics = ['acc'])
```

```
In [44]: model.fit(X, y, epochs=150, batch_size=32)
```

```
Epoch 44/150  
569/569 [=====] - 0s 141us/step - loss: 0.1971 - acc: 0.9209  
Epoch 45/150  
569/569 [=====] - 0s 129us/step - loss: 0.1959 - acc: 0.9192  
Epoch 46/150  
569/569 [=====] - 0s 138us/step - loss: 0.1863 - acc: 0.9315  
Epoch 47/150  
569/569 [=====] - 0s 131us/step - loss: 0.1980 - acc: 0.9104  
Epoch 48/150  
569/569 [=====] - 0s 288us/step - loss: 0.1922 - acc: 0.9244  
Epoch 49/150  
569/569 [=====] - 0s 124us/step - loss: 0.1949 - acc: 0.9262  
Epoch 50/150  
569/569 [=====] - 0s 178us/step - loss: 0.2065 - acc: 0.9121  
Epoch 51/150  
569/569 [=====] - 0s 257us/step - loss: 0.1997 - acc: 0.9262
```

Deep Learning Model Life-Cycle

❑ Fitting (training) the model

- ❑ The `fit` method on a Keras Model returns a **History** object.
- ❑ The **History.history** attribute is a dictionary recording training loss values and metrics values at successive epochs, as well as validation loss values and validation metrics values (if applicable).

```
history = model.fit(X_train, y_train, epochs=150, batch_size=32, verbose = 0)
```

```
history.history['loss']
```

```
[0.24846035641009415,  
 0.2507687051856299,  
 0.23816360427638678,  
 0.23443193424717962,  
 0.23879984811222146,  
 0.24287215006012175
```

Deep Learning Model Life-Cycle

□ The 5-Step Model Life-Cycle

- Define the model
- Compile the model
- Fit (train) the model
- Evaluate the model
- Make predictions

Deep Learning Model Life-Cycle

❑ Evaluating the model

- ❑ Evaluation of the model requires a dataset that was not used in the training process so that we can get unbiased estimate of the performance of the model when making predictions on new data.
- ❑ The evaluation of the model is much faster than the training since the model stays unchanged.

Deep Learning Model Life-Cycle

❑ Evaluating the model

- ❑ We can evaluate a model by calling the `evaluate` method with the test data set and getting a loss or other metric.

```
loss, accuracy = model.evaluate(x_test, y_test, verbose=0)  
print('Accuracy: %.2f' % (accuracy*100))
```

Accuracy: 66.93

Deep Learning Model Life-Cycle

□ The 5-Step Model Life-Cycle

- Define the model
- Compile the model
- Fit (train) the model
- Evaluate the model
- Make predictions

Deep Learning Model Life-Cycle

❏ Making predictions

- ❏ Once the model is trained and evaluated, it can be used to make predictions on new data, e.g. where you don't have the target values. This is done by calling the **predict** method.

```
1 predictions = model.predict(x_test, verbose=0)
2 for i in range(5):
3     print('%.3f'%predictions[i])
```

```
0.656
0.173
0.201
0.623
0.198
```


Deep Learning Model Life-Cycle

❏ Making predictions

❏ Alternatively, we can predict crisp classes directly.

```
1 pred_class = (model.predict(x_test, verbose=0) > 0.5).astype("int32")
2 for i in range(5):
3     print(pred_class[i])
```

```
[1]
[0]
[0]
[1]
[0]
```

Deep Learning Model Life-Cycle

Keras – MLP for binary classification

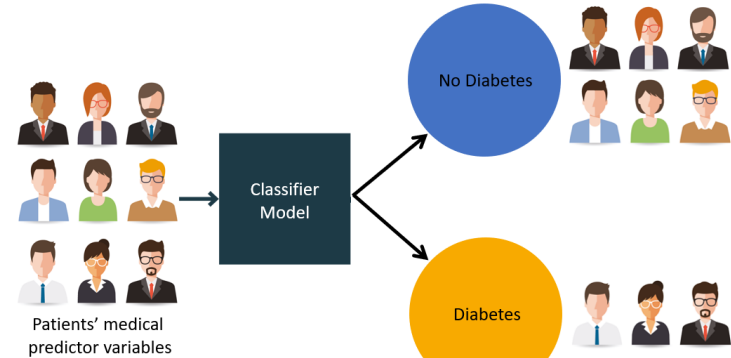


Deep Learning Model Life-Cycle

The objective is to build a predictive machine learning model to predict based on diagnostic measurements whether a patient has diabetes. This is a binary (2-class) classification problem.

The 8 medical predictor features are:

- **Pregnancies**
- **Glucose:**
- **Blood Pressure**
- **Skin Thickness**
- **Insulin**
- **BMI**
- **Diabetes Pedigree Function**
- **Age**



Coming next:

Analysing performance of ANNs